Recruitment and HR Resume Management System for Efficient Hiring

S Sai Rishik
Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad-500043, Telangana, India.
2420030121@klh.edu.in

G Chaitanya
Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad-500043, Telangana, India.
2420030721@klh.edu.in

J Vaman
Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad-500043, Telangana, India.
2420030736@klh.edu.in

Dr. Venkateswari Mam
Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad-500043, Telangana, India.

***Abstract*—Recruitment teams frequently screen resumes manually, store candidate information in disconnected files, and compare applicants with job descriptions without a consistent or transparent procedure. This paper presents a web-based Recruitment and HR Resume Management System that centralizes resume processing, candidate management, job posting, candidate ranking, shortlisting, application tracking, and recruitment notifications. The system follows a serviceoriented architecture implemented as six Spring Boot microservices that expose REST application programming interfaces (APIs) to an HTML5, CSS3, and JavaScript frontend. Resumes in PDF, DOCX, and TXT formats are converted to plain text using Apache PDFBox, Apache POI, and UTF-8 decoding. The principal contribution is an explainable resume– job matching module that detects technical skills using a predefined skill dictionary, alias normalization, and boundaryaware text matching, and computes the match percentage as the proportion of a job's required skills found in the candidate's resume or profile. Instead of reporting only a score, the module lists the matched and missing skills, generates a human-readable explanation of the score, and produces skill improvement recommendations. In the reported verification run, the multimodule Maven build completed successfully and all nine matching-engine unit tests, which cover 0%, 60%, 75%, and 100% match scenarios, passed. Empirical evaluation on realworld recruitment data is identified as future work.**

***Index Terms*—Applicant tracking system, explainable matching, microservices, recruitment management, resume parsing, skill gap analysis, Spring Boot**

I. Introduction

Recruitment is one of the most resource-intensive activities in human resource (HR) management. For every open position, recruiters receive resumes in heterogeneous formats, extract the relevant information, compare each applicant with the stated job requirements, and track the progress of each application through several hiring stages. When these steps are carried out manually or with disconnected tools such as spreadsheets and e-mail folders, screening becomes slow, candidate information becomes fragmented, and different reviewers may assess the same resume inconsistently. Applicant Tracking Systems (ATSs) address these problems by digitizing the hiring pipeline and centralizing candidate records [1].

Recent research has extended recruitment software with Natural Language Processing (NLP) and Machine Learning (ML) to automate resume parsing, classification, and recommendation [2], [3], and more recently with large language model (LLM) agents that summarize and grade resumes [4]. Although these techniques can accelerate screening, many of them output a ranking or a label without exposing the reasoning behind it. In hiring, this opacity is both a practical and an ethical concern: recruiters must be able to justify why a candidate was shortlisted or rejected, and studies of algorithmic hiring have emphasized the risk of bias and the need for transparency and human oversight [5], [6]. Research on interpretability similarly argues that decision-support systems should provide explanations that a human user can understand and verify [7], [8].

This paper presents a Recruitment and HR Resume Management System for Efficient Hiring, developed as an academic project. The system integrates the core recruitment workflow—resume management, candidate management, job posting, ranking, shortlisting, application tracking, and notifications—into a single web application built on a microservice architecture [9]. Its distinguishing feature is an Explainable Resume–Job Match and Skill Gap Analysis module. Instead of an opaque predictive model, the module uses a transparent dictionary-based and rule-based procedure: it detects skills in the candidate's resume and profile, normalizes them to canonical names, compares them with the skills required by a job posting, and reports the match percentage together with the matched skills, the missing skills, a plain-language explanation, and recommendations for closing each skill gap.

A. Problem Statement

Given a collection of candidate resumes in common document formats and a set of job postings with explicitly listed

required skills, an HR team needs (i) a single system in which resumes, candidates, jobs, and applications are stored and managed; (ii) a consistent and repeatable method for comparing each candidate with each job; and (iii) an output that explains, in terms the recruiter can verify, why a candidate received a particular score and which requirements remain unsatisfied. The comparison must support, rather than replace, the recruiter's judgement.

B. Research Gap

The literature reviewed in Section II shows two largely separate lines of work. ATS-oriented studies concentrate on workflow management and automated screening [1], [4], whereas person–job fit studies concentrate on the predictive quality of matching models, frequently using neural architectures trained on historical data [10], [11]. Less attention has been given to combining a complete recruitment workflow with a matching method in which every output can be traced back to explicit skills. The present work addresses this gap by embedding a transparent skill gap explanation directly in the recruitment workflow.

C. Contributions

The contributions of this paper are as follows:

1) The design and implementation of a recruitment management system composed of six Spring Boot microservices for resumes, jobs, candidates, applications, notifications, and matching, accessed by a web frontend through REST APIs.
2) A resume text extraction pipeline that supports PDF, DOCX, and TXT files.
3) An explainable, dictionary-based resume–job matching method with skill normalization, alias recognition, and boundary-aware matching, formalized in Section III-E.
4) An explainable skill gap analysis that presents matched skills, missing skills, a natural-language justification of the score, and skill improvement recommendations, integrated with candidate ranking, shortlisting, and application tracking.

The remainder of the paper is organized as follows. Section II reviews related work. Section III describes the architecture and methodology. Section IV presents the implementation results and the testing outcomes. Section V concludes the paper and outlines future work.

II. Literature Survey

In this section, prior work on automated recruitment, resume screening, skill extraction, person–job matching, fairness and explainability in algorithmic hiring, and microservice-based system design is reviewed briefly.

A. Applicant Tracking and Automated Resume Screening

Chavan *et al.* describe an advanced ATS intended to improve recruitment efficiency [1]. The system applies NLPbased resume parsing and classification to support candidate screening and the management of hiring activities. The work demonstrates the value of digitizing the recruitment pipeline; its emphasis, however, is on ATS functionality rather than on explaining how an individual candidate compares with the requirements of a specific job.

Gan, Zhang, and Mori propose a framework in which LLM agents perform automated resume screening [4]. A fine-tuned LLaMA2 model first classifies resume sentences into predefined categories, personal information is removed, and an LLM-based agent then summarizes and grades each resume and supports the decision on which candidates proceed. The authors report that the framework was 11 times faster than manual screening and that fine-tuning yielded an F1 score of 87.73% for resume sentence classification. The approach illustrates the potential of generative models for screening at scale, but it relies on large models whose grading rationale is not inherently transparent, and it focuses on screening rather than the complete recruitment workflow.

Roy, Chowdhary, and Bhatia present an ML approach for automating resume recommendation [3]. After classifying resumes into job categories, their system uses cosine similarity and the k-nearest neighbours (k-NN) method to identify the resumes closest to a given job description. Sinha, Akhtar, and Kumar provide a systematic review of NLP and ML techniques for resume screening [2], summarizing the parsing, feature extraction, and classification methods reported in the literature. Both studies show that similarity- and classificationbased screening can narrow a candidate pool effectively, while a similarity value alone does not tell the recruiter which requirements a candidate satisfies.

Faliagka, Tsakalidis, and Tzimas present an integrated erecruitment system that ranks applicants using criteria extracted from their LinkedIn profiles together with personality traits inferred through linguistic analysis [12]. The analytic hierarchy process (AHP) is used to derive the ranking, and the relative weights of the selection criteria are controlled by the recruiter. Keeping the recruiter in control of the final assessment is a principle that is also adopted in the present system, where the match score is presented strictly as decision support.

B. Person–Job Fit and Skill Extraction

Zhu *et al.* formulate person–job fit as a joint representation learning problem and propose a convolutional neural network that maps job postings and resumes into a shared latent space [10]. Qin *et al.* propose an ability-aware neural network for person–job fit that uses attention mechanisms to relate the abilities required by a job to the experience described in a resume [11]. These models learn from historical recruitment records and can capture semantic relationships beyond exact skill names, but they require large labelled datasets, and their scores are harder for a recruiter to verify.

Zhang *et al.* introduce SkillSpan, a dataset of English job postings annotated with hard and soft skill spans, and evaluate transformer-based models, including domain-adapted variants, for span-level skill extraction [13]. Their study shows that skill extraction from free text is non-trivial because skills are expressed in varied and sometimes lengthy phrases. This

observation motivates the explicit normalization and alias handling used in the present work, and it also clarifies the main limitation of a dictionary-based extractor, which can recognize only the skills it has been configured to detect.

C. Fairness, Ethics, and Explainability in Hiring

Raghavan *et al.* examine the claims and practices of vendors of algorithmic pre-employment assessments and analyze how bias is detected, mitigated, and disclosed [5]. Fabris *et al.* present a multidisciplinary survey of fairness and bias in algorithmic hiring that covers sources of bias, fairness measures, and the legal and organizational context [6]. Mujtaba and Mahapatra discuss ethical considerations in AIbased recruitment, including bias, transparency, and accountability [14]. Together, these works argue that automated hiring tools should be transparent, auditable, and subject to human oversight.

Guidotti *et al.* survey methods for explaining black-box models [8], and Adadi and Berrada review the broader field of explainable artificial intelligence (XAI) [15]. Doshi-Velez and Kim argue for a rigorous definition and evaluation of interpretability [7]. A recurring theme in this literature is the distinction between post-hoc explanations of opaque models and models that are interpretable by design. The matching method in this paper belongs to the second category: because the score is computed directly from explicit skill sets, the explanation is an exact description of the computation rather than an approximation of it.

D. Microservice-Based Web Applications

Dragoni *et al.* review the evolution of microservices and characterize them as small, independently deployable services that communicate through lightweight mechanisms [9]. Fielding's Representational State Transfer (REST) architectural style [16] provides the stateless, resourceoriented interface model that is widely used for communication between web clients and such services. These principles guide the decomposition of the proposed system into separate services, one for each recruitment responsibility.

E. Summary of the Gap

The reviewed studies demonstrate (i) mature workflow support in ATS platforms, (ii) accurate but less transparent learning-based matching, and (iii) strong arguments for transparency in automated hiring. The present work does not claim higher matching accuracy than learning-based approaches. Its contribution is an integrated recruitment system in which every match score is accompanied by the exact matched and missing skills that produced it, together with actionable recommendations.

III. Methodology

This section describes the system architecture, the end-toend workflow, and each processing stage, including the formal definition of the matching score. It concludes with the implementation stack and the testing environment.

A. System Architecture

The system follows a service-oriented architecture implemented with microservices, as shown in Fig. 1. Each recruitment responsibility is implemented as a separate Spring Boot [17] application with its own REST endpoints, service logic, and persistence layer based on Spring Data JPA [18]. The six services run on separate ports, as summarized in Table I, and are built together as a multi-module Apache Maven [19] project. The frontend is a browser-based application written in HTML5, CSS3, and JavaScript; it invokes the services through HTTP REST endpoints and renders the returned JSON data in the HR dashboard and the module views. In the current demonstration configuration, data is stored in the H2 relational database engine [20], and an SQL schema describing the recruitment entities is provided with the project.

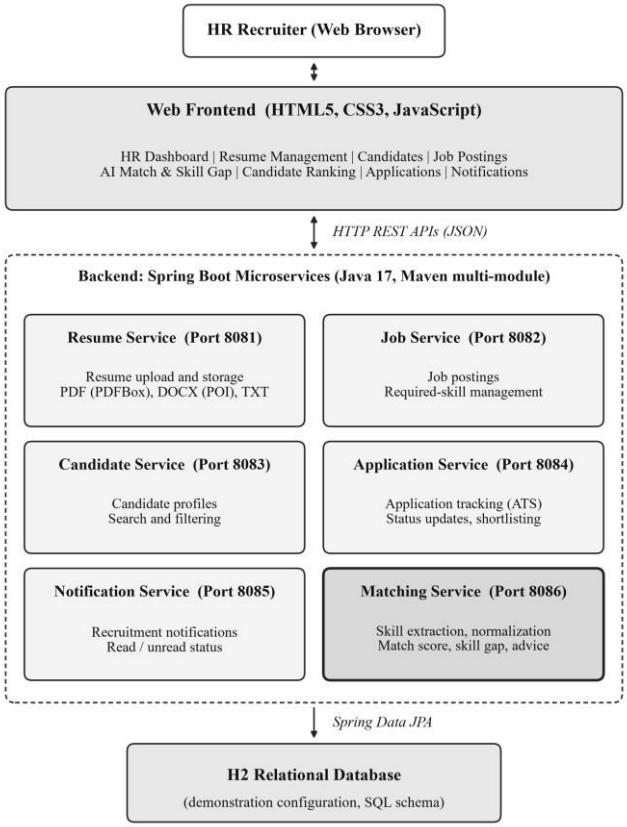


Fig. 1. System architecture of the Recruitment and HR Resume Management System: the web frontend communicates through REST APIs with six Spring Boot microservices, which persist data in the H2 database used in the demonstration configuration.

TABLE I
BACKEND MICROSERVICES AND THEIR RESPONSIBILITIES

Service	Port	Main Responsibilities
Resume Service	8081	Resume upload, storage, and text extraction from PDF, DOCX, and TXT files
Job Service	8082	Creation and management of job postings and their required skills
Candidate Service	8083	Candidate profiles, candidate search, and filtering

Application Service	8084	Application tracking, status updates, and shortlisting
Notification Service	8085	Recruitment notifications and read/unread status
Matching Service	8086	Skill extraction, normalization, match scoring, skill gap analysis, and recommendations

Separating responsibilities in this way has several practical benefits. Each service can be developed, tested, and started independently; a change in one module, such as the matching logic, does not require modification of unrelated modules such as notifications; and each service exposes a small, welldefined interface. Because the frontend interacts with the services only through their REST interfaces, the internal implementation of a service can evolve without affecting the user interface. The present implementation is, however, a demonstration configuration: an API gateway, service discovery, centralized authentication, and container orchestration are not part of the implemented system and are discussed as future work in Section V.

B. System Workflow

The complete recruitment workflow implemented by the system consists of the following stages:

1) *Resume upload:* the recruiter uploads a resume file or enters resume text and optionally associates it with a candidate.
2) *Text extraction:* the Resume Service converts the document to plain text.
3) *Skill extraction:* the Matching Service detects dictionary skills in the candidate's resume text and profile.
4) *Skill normalization:* detected terms and supported aliases are mapped to canonical skill names.
5) *Job requirement comparison:* the candidate's skill set is compared with the required skills of the selected job posting.
6) *Match score calculation:* the match percentage is computed using (5).
7) *Skill gap identification:* required skills that were not detected are reported as missing.
8) *Explanation and recommendations:* a textual explanation and per-skill recommendations are generated.
9) *Candidate ranking:* candidates are ordered by their match percentage for a job.
10) *Shortlisting and application tracking:* the recruiter reviews the results and updates the application status, and the system records recruitment notifications.

At every stage, the final decision remains with the recruiter; the score is presented as a decision-support signal and not as an automatic selection decision.

C. Resume Text Extraction

The Resume Service accepts resumes in three formats, as indicated in the resume management interface described in Section IV-A. PDF files are processed with Apache PDFBox [21], which extracts the text content of the document pages. DOCX files are processed with Apache POI [22], which reads the paragraphs of Office Open XML word-processing documents. TXT files are decoded using the UTF-8 character encoding. The extracted plain text is stored with the resume record and can be associated with a candidate profile. Alternatively, the recruiter can enter resume text directly through a text-entry form, which bypasses file parsing but follows the same downstream processing. Automated tests for resume parsing are part of the Maven test suite.

Text extraction from a PDF document depends on the presence of an embedded text layer. Scanned resumes that contain only images would require optical character recognition (OCR), which is not part of the current implementation. Complex multi-column layouts may also yield text in an order that differs from the visual reading order; this has limited effect on the matching method, which depends on the presence of skill terms rather than on their position.

D. Skill Extraction and Normalization

Skill extraction is performed by the Matching Service using a predefined technical skill dictionary. Each dictionary entry consists of a canonical skill name, such as Java, Spring Boot, SQL, REST API, React, or Node.js, and, where supported, a set of aliases that represent alternative spellings or abbreviations of the same skill. Let D denote the dictionary, T_C the resume text of candidate C, and P_C the skills recorded in the candidate's profile. The extraction function $E(\cdot, D)$ returns the dictionary terms that occur in its input, and the normalization function $N(\cdot)$ maps every detected term or alias to its canonical name. The candidate skill set is then

$$S_C = N(E(T_C, D)) \cup N(P_C) \quad (1)$$

Similarly, the required skills of job J, entered by the recruiter in the Job Service as a list K_J, are normalized to

$$R_J = N(K_J) \quad (2)$$

Normalization ensures that superficial differences in how a skill is written, including supported alternative names, do not cause a required skill to be reported as missing. Text matching is boundary-aware: a dictionary term is accepted only when it appears as a complete term and not as a fragment of a longer word. This prevents, for example, the skill Java from being detected inside the word JavaScript, and it requires skill names containing punctuation, such as Node.js, to be matched as complete terms. Automated tests for skill extraction are included in the test suite.

Because the extractor detects only dictionary entries and their supported aliases, its coverage is determined entirely by the dictionary. A skill written with unsupported terminology, or a related technology that is not registered as an alias, will not be recognized. This trade-off is deliberate: the method gives up coverage of unseen terminology in exchange for outputs that are fully predictable and explainable.

E. Resume Job Matching Algorithm

Given S_C and R_J, the set of matched skills and the set of missing skills (the skill gap) are defined as

$$M_{C,J} = R_J \cap S_C \tag{3}$$

$$G_{C,J} = R_J \setminus S_C \tag{4}$$

and the match percentage is defined as

$$\text{Score}(C,J) = \frac{|M_{C,J}|}{|R_J|} \times 100 \tag{5}$$

where $|\cdot|$ denotes set cardinality, R_J is the set of normalized required skills of job J, S_C is the set of normalized skills detected for candidate C, $M_{C,J}$ is the set of required skills found in the candidate's resume or profile, $G_{C,J}$ is the set of required skills that were not found, and Score(C,J) ∈ [0, 100] is the match percentage. By construction, $|M_{C,J}| + |G_{C,J}| = |R_J|$, so every required skill is reported exactly once, either as matched or as missing. The formula assumes that the job posting lists at least one required skill ($|R_J| \geq 1$). Skills that the candidate possesses but the job does not require do not change the score, because the measure expresses coverage of the job's requirements rather than overall candidate quality. In the current formula, all required skills carry equal weight. The complete procedure is summarized in Algorithm 1.

Algorithm 1: Explainable Resume–Job Matching

Input: resume text T_C, profile skills P_C, required skills K_J, dictionary D
Output: Score, matched set M, missing set G, explanation X, recommendations Rec

$S_C \leftarrow N(E(T_C, D)) \cup N(P_C)$
$R_J \leftarrow N(K_J)$
$M \leftarrow \emptyset$; $G \leftarrow \emptyset$
for each skill $r \in R_J$ *do*
    if $r \in S_C$ *then* $M \leftarrow M \cup \{r\}$
    else $G \leftarrow G \cup \{r\}$
end for
Score ← ($|M| / |R_J|$) × 100
X ← statement of $|M|$, $|R_J|$, Score, M, and G
Rec ← one recommendation for each skill in G
return Score, M, G, X, Rec

Worked example: Consider the demonstration case presented in Section IV-A, in which a sample candidate profile is matched against a Full Stack Web Developer posting. The posting requires eight skills: R_J = {React, JavaScript, HTML, CSS, Node.js, SQL, REST API, Git}. The candidate profile lists Java, Spring Boot, SQL, REST API, Microservices, Git, Docker, MySQL, and PostgreSQL. The matched set is therefore $M_{C,J}$ = {Git, REST API, SQL}, and the skill gap is $G_{C,J}$ = {CSS, HTML, JavaScript, Node.js, React}. Applying (5),

$$\text{Score} = \frac{3}{8} \times 100 = 37.5\% \tag{6}$$

The explanation reports the value to one decimal place (37.5%), while the circular score indicator displays the rounded value 38% together with a qualitative fit label. The example also illustrates the requirement-coverage property of the score: the candidate's backend skills, such as Spring Boot and Docker, are genuine strengths but do not raise the score for a posting that does not require them.

F. Explainable Skill Gap Analysis and Recommendations

The explanation is generated directly from the quantities computed in Algorithm 1. For the example above, the system states that the candidate matches 3 of the 8 required skills (37.5%), names the detected skills, and lists the missing ones. Because the explanation enumerates exactly the sets used in (5), the recruiter can verify the score by inspection, and the explanation is faithful to the computation by construction—a property that post-hoc explanations of black-box models do not guarantee [8].

For each skill in $G_{C,J}$, the system generates a skill improvement recommendation. In the implemented output, missing skills for which specific guidance is available receive targeted advice (for example, practising React functional components and hooks, or building asynchronous services with Node.js), while other missing skills receive a general recommendation to practise hands-on projects and tutorials for that skill. The recommendations serve two audiences: recruiters can use them to understand the nature of a gap, and candidates can use them as a learning plan.

G. Candidate Ranking, Shortlisting, and Application Tracking

For a selected job posting, candidates are ranked in descending order of Score(C,J), producing a job-specific leaderboard that is accessible from the Candidate Ranking view. Ranking by requirement coverage gives recruiters a consistent starting point for review, while the accompanying skill lists allow them to distinguish, for instance, a candidate who lacks one critical skill from a candidate who lacks several minor ones—a distinction that the equal-weight score itself does not capture.

The Application Service implements the ATS functionality. Each application links a candidate to a job posting and carries a status that represents its recruitment stage, including shortlisted and selected states. Recruiters update the status after reviewing the match results, and the Notification Service records recruitment-related notifications, which are displayed with read/unread status in the Notifications view. The Candidate Service supports searching and filtering of candidate profiles, and the dashboard aggregates the numbers of candidates, open positions, processed resumes, and tracked applications.

H. Implementation Details and Testing Environment

The implementation stack is summarized in Table II. No external AI service, trained model, or third-party matching library is used by the matching module; skill detection, scoring, explanation, and recommendation are implemented as deterministic application logic in the Matching Service.

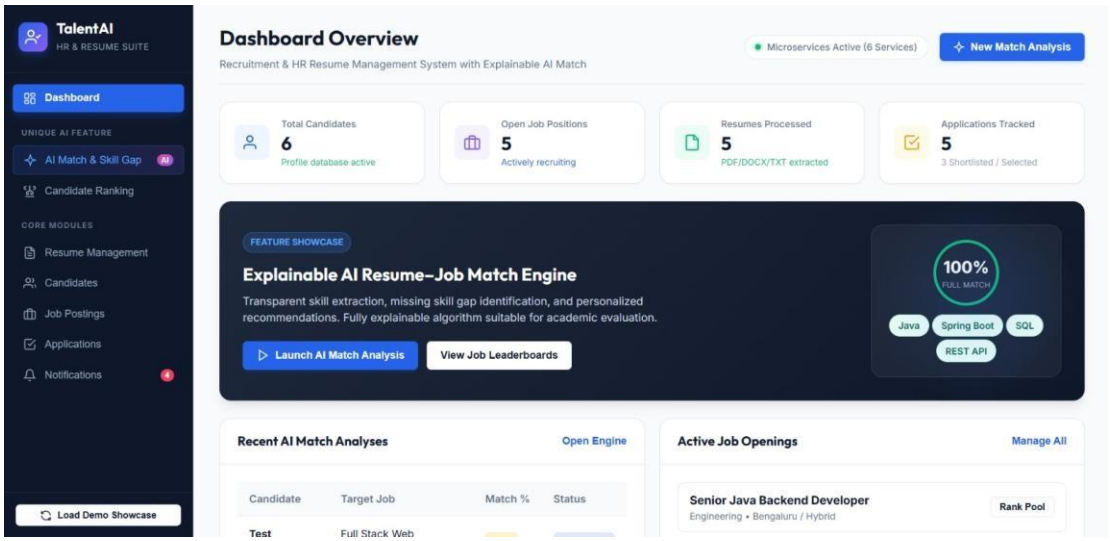


Fig. 2. HR dashboard showing recruitment summary statistics, the explainable matching feature panel, recent match analyses, and active job openings (demonstration data).

TABLE II
IMPLEMENTATION STACK

Layer	Technology
Frontend	HTML5, CSS3, JavaScript (served on port 5500 for demonstration)
Backend	Java 17, Spring Boot, REST APIs
Persistence	Spring Data JPA, H2 relational database, SQL schema
Resume parsing	Apache PDFBox (PDF), Apache POI (DOCX), UTF-8 decoding (TXT)
Build	Apache Maven (multi-module project)
Testing	JUnit-based unit tests executed through Maven

The system was built and tested with the Maven test lifecycle (*mvn clean test*), which compiles all modules and executes the JUnit-based unit tests [23]. For the runtime demonstration, the six backend services were started on ports 8081–8086 and the frontend was served on port 5500. The verification was performed on a system with the following specifications:

- CPU: `[processor model — to be filled]`
  - RAM: only the heading appears.
  - OS: only the heading appears.
  - Browser: only the heading appears.

IV. RESULTS & ANALYSIS

This section presents the implemented user interface and the results of automated testing. The evaluation reported here verifies the functional behaviour of the implementation; it is not an empirical measurement of screening accuracy on realworld recruitment data.

A. Implemented User Interface

Fig. 2 shows the HR dashboard. Four summary cards report the totals of candidates, open job positions, processed resumes,

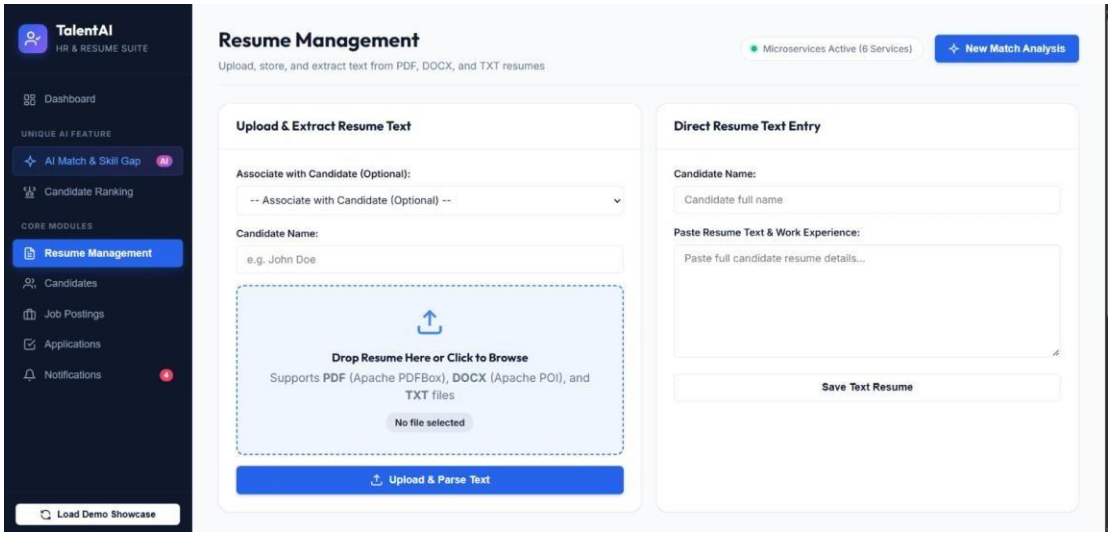


Fig. 3. Resume management interface for uploading and parsing PDF, DOCX, and TXT resumes and for direct entry of resume text.

and tracked applications, including the number of shortlisted or selected applications. A feature panel provides direct access to the matching engine and to job leaderboards, and the lower panels list recent match analyses and active job openings. A status indicator reports that the six microservices are active. The values displayed in Fig. 2 originate from demonstration data loaded through the Load Demo Showcase function and are not measurements of real recruitment activity.

Fig. 3 shows the resume management interface. The left panel allows a resume file to be uploaded, optionally associated with an existing candidate, and parsed to text; the right panel supports direct entry of resume text. The interface states the supported formats and the library used for each format.

Fig. 4 shows the candidate and job selection step of the matching module. The recruiter selects a candidate profile and a target job opening, and the interface displays the candidate's education, experience, and recorded skills alongside the job's department, experience requirement, location, and required skills. Additional tabs allow matching against directly entered custom text and display the analysis history.

Fig. 5 shows the output of the matching module for the worked example of Section III-E. The score indicator, the matched-skill list, the skill gap list, the explainable score breakdown, and the recommendations are presented together, so that every element of the score can be traced to an explicit job requirement. The algorithm used is also stated in the interface, which allows a recruiter to recompute the score manually.

B. Testing and Validation

Testing was carried out at two levels. First, automated unit tests were executed as part of the Maven build. Second, the running services and the frontend were exercised manually for demonstration. Table III summarizes the reported verification results, and Table IV lists the score scenarios covered by the matching-engine unit tests.

TABLE III
SUMMARY OF REPORTED VERIFICATION RESULTS

Verification activity	Type	Reported outcome
Multi-module Maven build (mvn clean test)	Automated	Completed successfully
Matching-engine unit tests	Automated (JUnit)	9 of 9 passed
Skill extraction tests	Automated (JUnit)	Passed within successful build
Resume parsing tests	Automated (JUnit)	Passed within successful build
Six services on ports 8081–8086	Manual runtime check	Started and checked for demonstration
Frontend views (Figs. 2–5)	Manual demonstration	Views operated with demonstration data
Automated end-to-end tests	—	Not performed in this work

TABLE IV
SCORE SCENARIOS COVERED BY THE MATCHING-ENGINE UNIT TESTS

Scenario	Expected score	Example ratio $\lvert M\rvert / \lvert R\rvert$	Result
No required skill detected	0%	0 / n	Passed
Partial match	60%	e.g., 3 / 5	Passed
Partial match	75%	e.g., 3 / 4	Passed
All required skills detected	100%	n / n	Passed

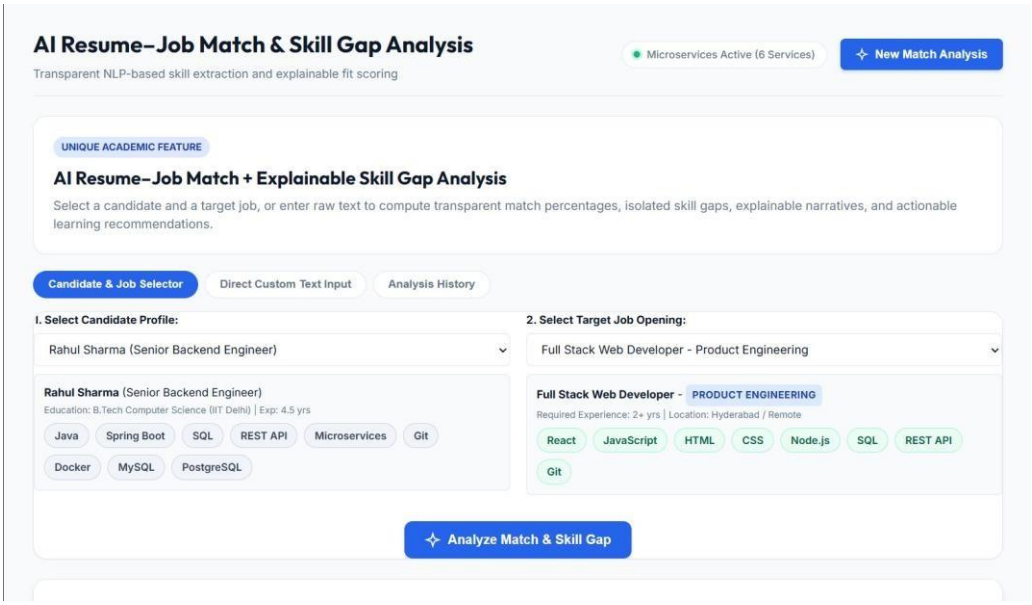


Fig. 4. Candidate and job selection interface of the resume–job matching module, showing the candidate's recorded skills and the job's required skills.

The example ratios in Table IV are illustrative values that produce the stated scores; the exact test fixtures are defined in the project's test code. The scenarios exercise both boundaries of the score (0% and 100%) and two intermediate values, confirming that (5) is implemented as specified and that each required skill is classified as either matched or missing. Because the score is a deterministic function of the normalized skill sets, identical inputs always produce identical scores, explanations, and recommendations; this repeatability follows directly from the rule-based design.

The unit tests verify individual components in isolation, principally the matching engine, skill extraction, and resume parsing. They do not verify the interaction between services over HTTP, the behaviour of the frontend, or the persistence of data across restarts. These aspects were checked manually during the runtime demonstration, and automated integration and end-to-end testing remain future work. No timing, accuracy, precision, recall, or user-study measurements were collected in the current work, and no such results are claimed.

C. Discussion: Benefits and Limitations

The principal strength of the method is transparency. As the worked example shows, a recruiter can reconstruct the score of 37.5% directly from the listed skills. The method requires no training data and no external AI service, which makes it inexpensive to run and simple to audit, and its behaviour can be changed predictably by editing the dictionary or the required skills of a job. Its deterministic nature also supports consistent treatment of candidates: two candidates with the same detected skills receive the same score. Since the score is computed only from skill sets, personal attributes such as a candidate's name are not inputs to (5); this does not by itself guarantee fairness, because bias can still enter through the choice of required skills or through differences in how candidates describe their experience [5],
[6].

The current approach also has the following limitations:

1) *Dictionary dependence:* only skills present in the predefined dictionary and its supported aliases can be detected.

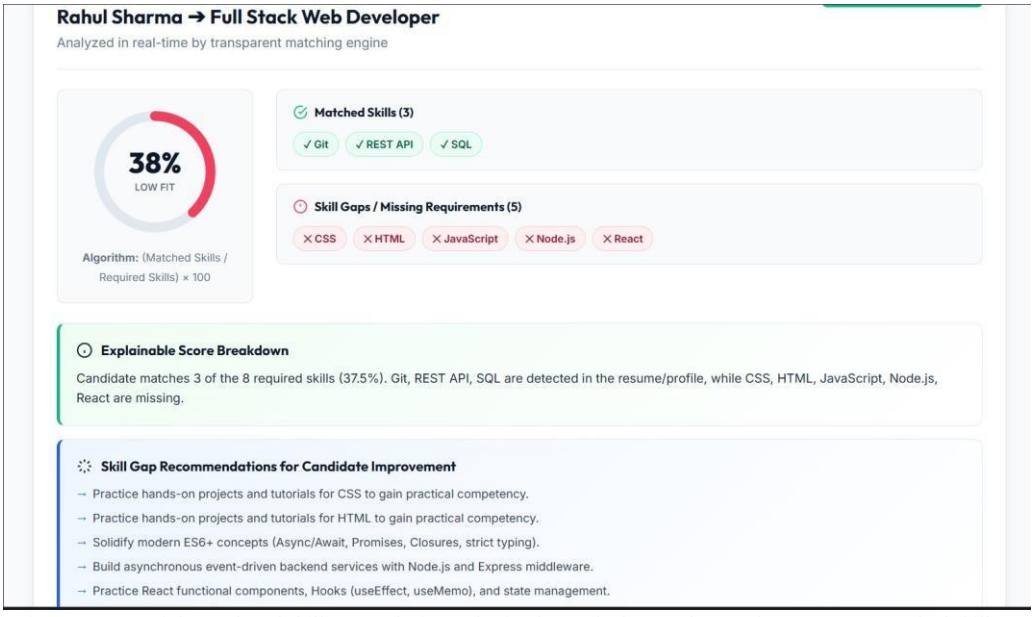


Fig. 5. Resume–job match and skill gap analysis results for the worked example: match percentage, matched skills, missing skills, explainable score breakdown, and skill improvement recommendations.

2) *Mention versus proficiency:* the presence of a skill term does not demonstrate proficiency, depth, or recency of experience; a single mention counts the same as years of practical use.
3) *Equal weighting:* every required skill contributes equally, so missing a core skill lowers the score by the same amount as missing a peripheral one.
4) *Semantic equivalence:* equivalent or closely related technologies described with unsupported terminology may not be recognized, and skills implied only by context cannot be inferred.
5) *Limited evaluation:* the evaluation verifies implementation behaviour through unit tests and demonstration; accuracy and practical effectiveness on real-world resumes have not yet been measured.
6) *Deployment scope:* the system uses H2 in a demonstration configuration and does not yet implement authentication; it is therefore a working prototype rather than a production-ready system.

V. Conclusion

This paper presented a Recruitment and HR Resume Management System for Efficient Hiring that integrates resume management, candidate management, job posting, candidate ranking, shortlisting, application tracking, and notifications in a web application built from six Spring Boot microservices. Resumes in PDF, DOCX, and TXT formats are converted to text, and an explainable resume–job matching module compares the skills detected in a candidate's resume and profile with the skills required by a job posting. Instead of an opaque score, the module reports the match percentage together with the matched skills, the missing skills, a humanreadable explanation, and skill improvement recommendations, so that every score can be verified by the recruiter.

The reported verification showed that the multi-module Maven build completed successfully and that all nine matching-engine unit tests, covering 0%, 60%, 75%, and 100% match scenarios, passed. These results confirm that the matching method behaves as specified. They do not measure screening accuracy on real-world data, which remains an important direction for further work.

This work can be further extended in the following ways:

1) Introducing semantic matching, for example through skill ontologies or text embeddings, to recognize equivalent skills expressed with different terminology while still reporting which terms were matched.
2) Introducing weighted skill scoring, in which recruiters mark skills as mandatory or optional and assign weights, so that the score reflects the relative importance of each requirement.
3) Improving resume information extraction to capture experience duration, education, and project context, and adding OCR support for scanned resumes.
4) Adding secure authentication and role-based access control for HR users.
5) Configuring a persistent production database and deploying the services to the cloud with containerization, an API gateway, and service discovery.
6) Conducting an empirical evaluation on an annotated set of real resumes and job postings, measuring skill extraction quality and agreement with recruiter decisions, together with a fairness assessment and a user study with HR professionals.

References

[1] R. Chavan, S. Gaikwad, G. Lavankar, A. Tidake, Y. Chandurkar, and P. R. Chavan, "Enhancing recruitment efficiency: An advanced applicant tracking system (ATS)," *Ind. Manage. Adv.*, vol. 2, no. 1, Art. no. 6373, 2024, doi: 10.59429/ima.v2i1.6373.

[2] A. K. Sinha, M. A. K. Akhtar, and A. Kumar, "Resume screening using natural language processing and machine learning: A systematic review," in *Machine Learning and Information Processing* (Advances in Intelligent Systems and Computing). Singapore: Springer, 2021, pp. 207–214, doi: 10.1007/978-981-33-4859-2_21.

[3] P. K. Roy, S. S. Chowdhary, and R. Bhatia, "A machine learning approach for automation of resume recommendation system," *Procedia Comput. Sci.*, vol. 167, pp. 2318–2327, 2020, doi: 10.1016/j.procs.2020.03.284.

[4] C. Gan, Q. Zhang, and T. Mori, "Application of LLM agents in recruitment: A novel framework for automated resume screening," *J. Inf. Process.*, vol. 32, pp. 881–893, 2024, doi: 10.2197/ipsjjip.32.881.

[5] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating bias in algorithmic hiring: Evaluating claims and practices," in *Proc. Conf. Fairness, Accountability, Transparency (FAT* , Barcelona, Spain, 2020, pp. 469–481, doi: 10.1145/3351095.3372828.

[6] A. Fabris, N. Baranowska, M. J. Dennis, D. Graus, P. Hacker, J. Saldivar, F. Zuiderveen Borgesius, and A. J. Biega, "Fairness and bias in algorithmic hiring: A multidisciplinary survey," *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 1, pp. 1–54, 2025, doi: 10.1145/3696457.

[7] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," arXiv:1702.08608, 2017.

[8] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Comput. Surv.*, vol. 51, no. 5, pp. 1–42, 2018, doi: 10.1145/3236009.

[9] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds. Cham, Switzerland: Springer, 2017, pp. 195–216, doi: 10.1007/978-3-319-67425-4_12.

[10] C. Zhu, H. Zhu, H. Xiong, C. Ma, F. Xie, P. Ding, and P. Li, "Personjob fit: Adapting the right talent for the right job with joint representation learning," *ACM Trans. Manage. Inf. Syst.*, vol. 9, no. 3, pp. 1–17, 2018, doi: 10.1145/3234465.

[11] C. Qin, H. Zhu, T. Xu, C. Zhu, L. Jiang, E. Chen, and H. Xiong, "Enhancing person-job fit for talent recruitment: An ability-aware neural network approach," in *Proc. 41st Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2018, pp. 25–34, doi: 10.1145/3209978.3210025.

[12] E. Faliagka, A. Tsakalidis, and G. Tzimas, "An integrated e-recruitment system for automated personality mining and applicant ranking," *Internet Res.*, vol. 22, no. 5, pp. 551–568, 2012, doi: 10.1108/10662241211271545.

[13] M. Zhang, K. Jensen, S. Sonniks, and B. Plank, "SkillSpan: Hard and soft skill extraction from English job postings," in *Proc. 2022 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol. (NAACL-HLT)*, Seattle, WA, USA, 2022, pp. 4962–4984, doi: 10.18653/v1/2022.naacl-main.366.

[14] D. F. Mujtaba and N. R. Mahapatra, "Ethical considerations in AIbased recruitment," in *Proc. IEEE Int. Symp. Technol. Soc. (ISTAS)*, 2019, pp. 1–7, doi: 10.1109/ISTAS48451.2019.8937920.

[15] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on explainable artificial intelligence (XAI)," *IEEE Access*, vol. 6, pp.

52138–52160, 2018, doi: 10.1109/ACCESS.2018.2870052.
[16] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.
[17] Spring, “Spring Boot reference documentation.” [Online]. Available: https://docs.spring.io/spring-boot/ (accessed Oct. 30, 2026).
[18] Spring, “Spring Data JPA reference documentation.” [Online]. Available: https://docs.spring.io/spring-data/jpa/reference/ (accessed Oct. 30, 2026).
[19] The Apache Software Foundation, “Apache Maven project.” [Online]. Available: https://maven.apache.org/ (accessed Oct. 30, 2026).
[20] “H2 database engine.” [Online]. Available: https://www.h2database.com/ (accessed Oct. 30, 2026).
[21] The Apache Software Foundation, “Apache PDFBox: A Java PDF library.” [Online]. Available: https://pdfbox.apache.org/ (accessed Oct. 30, 2026).
[22] The Apache Software Foundation, “Apache POI: The Java API for Microsoft documents.” [Online]. Available: https://poi.apache.org/ (accessed Oct. 30, 2026).
[23] “JUnit 5 user guide.” [Online]. Available: https://junit.org/junit5/docs/current/user-guide/ (accessed Oct. 30, 2026).